

Incident Response Engagement Findings Report

Mission 04 - SQL Injection and Credential Reuse

CLIENT

NexaCorp Industries

SQL Injection investigation, employee self-service portal

Engagement reference INC-2026-004

Target host: bru-web-01

Prepared by

Johan-Emmanuel Hatchi

BeCode Cybersecurity Bootcamp - Promotion 2025-2026

linkedin.com/in/johan-emmanuel-hatchi · github.com/Jhatchi

Date: 11 June 2026

Table of Contents

CONFIDENTIALITY AND AUTHORIZATION	3
DISCLAIMER	3
EXECUTIVE SUMMARY	4
A. Plain-language summary	4
B. Engagement summary	4
C. Scope and evidence	4
D. Overview of findings	4
E. Overview of recommendations	4
METHODOLOGY	7
A. Procedure	7
B. Severity Ratings	7
ATTACK TIMELINE	9
INVESTIGATION TECHNICAL SUMMARY	10
4.1 SQL Injection on the id parameter	11
4.2 Database enumeration and users table exfiltration	13
4.3 Weak credential recovered offline	15
4.4 Credential reuse, successful SSH foothold	16
INDICATORS OF COMPROMISE	18
REMEDATION RECOMMENDATIONS	19
DETECTION ENGINEERING	20
RISK MATRIX	21
MITRE ATT&CK MAPPING	22
TOOLS AND STANDARDS	23
GLOSSARY AND ACRONYMS	25

Confidentiality and Authorization

This report documents **Mission 04** of the BeCode Corp incident response training programme. The engagement investigates a SQL Injection against the fictitious organisation *NexaCorp Industries* and its employee self-service portal hosted on *bru-web-01*, deployed in the BeCode lab under reference **BCC-2026**.

No real production system, customer data, or third-party infrastructure was targeted, accessed, or affected by this engagement. All IP addresses, hostnames, and accounts referenced in this report are lab-local artifacts with no production analogue.

This document is classified **Confidential** under reference **BCC-2026 | INC-2026-004**. Publication for portfolio use was authorized by the BeCode lab coach (Thomas B.) on 17 May 2026. Subject to that authorization, the analyst retains the right to share this report as part of professional portfolios, certification evidence, and job applications, in line with the BeCode Corp policy on training deliverables.

Disclaimer

The findings and recommendations in this report are based on the evidence bundle provided for the incident: the web access log (`web_access.log`, 1342 requests), the network capture (`attack.pcap`), the SIEM export (`wazuh-alerts.json`), and the authentication log (`auth.log`). The analysis is bounded by what those sources record.

All timestamps use the web server local time zone (UTC+02:00), which matches the authoritative request record. The PCAP capture window is expressed in UTC: 2026-05-30 07:34:03Z to 14:14:55Z. The investigation applies the methodology recommended by **NIST SP 800-61r2** (Computer Security Incident Handling Guide) and the **SANS PICERL** process, and classifies the root vulnerability under **OWASP A03:2021 Injection**.

Executive Summary

A. Plain-language summary

Someone outside the company found a weak spot in the staff portal and used it to read the company database through the web page itself. They pulled out the list of user accounts and their scrambled passwords. One staff account, j.martin, had a password weak enough to be unscrambled in seconds on an ordinary laptop.

A few hours later, that same outsider used the recovered password to log in to the server directly, and it worked. They now have a working login on the server, using a real employee's identity. Every password stored in the portal should be considered exposed. The portal code needs a fix so this cannot happen again, and the compromised account must be locked immediately.

B. Engagement summary

An external attacker (172.16.50.10) exploited a SQL Injection vulnerability in the id parameter of the NexaCorp employee portal on bru-web-01. The attacker confirmed the flaw with an error-based probe, mapped the database, and exfiltrated the full users table, including six accounts and their MD5 password hashes. The employee account j.martin used a password recovered offline in under one second against a standard wordlist.

Approximately five hours later, the attacker reused that credential to authenticate over SSH to the same server and succeeded, then attempted to spray the other stolen accounts (which failed). The outcome is a confirmed authenticated foothold on bru-web-01 and the exposure of every credential in the portal database. Immediate remediation is required.

Findings by Severity Level

Critical		2
High		2
Medium		0
Low		0

B.

C. Scope and evidence

Forensic analysis of the provided evidence bundle, covering four phases: web access log analysis, PCAP response analysis, SIEM alert review, and consequence analysis. After the investigation reconstructed the attack, a detection engineering pass produced and validated four Suricata signatures (see the Detection Engineering section). Every finding is mapped to MITRE ATT&CK Enterprise v15.

Evidence sources: `web_access.log` (1342 requests total), `attack.pcap`, `wazuh-alerts.json`, `auth.log`.

D. Overview of findings

This list provides the general overview of all findings documented in this report. Findings are numbered 4.1 to 4.4 for parity with the published engagement repository.

Ref	Description	Severity
4.1	SQL Injection on the id parameter of the employee portal (UNION-based, full database)	CRITICAL
4.2	Database enumeration and full users table exfiltration (six accounts, MD5 hashes)	HIGH
4.3	Weak credential recovered offline (j.martin, unsalted MD5, instant crack)	HIGH
4.4	Credential reuse leading to a successful authenticated SSH foothold on bru-web-01	CRITICAL

E. Overview of recommendations

Short overview of the proposed recommendations by finding. Detailed, prioritised remediation is provided in the Remediation Recommendations section and alongside each finding.

<i>Ref</i>	<i>Recommendation</i>
4.1	Replace the query with parameterised statements; add server-side input validation on the id parameter.
4.2	Treat every account in the dumped table as compromised and force a reset; fix the root injection.
4.3	Block common and breached passwords; enforce MFA; migrate storage from unsalted MD5 to bcrypt or argon2.
4.4	Disable or reset j.martin immediately; enforce MFA on SSH; restrict SSH to internal or VPN; block the attacker IP.

Methodology

A. Procedure

This analysis applies industry-standard incident response frameworks tailored to a SOC environment. The methodology aligns with **NIST SP 800-61 Rev. 2** (Computer Security Incident Handling Guide) and the **SANS PICERL** process (*Preparation, Identification, Containment, Eradication, Recovery, Lessons learned*). The investigation ingests the evidence bundle, reconstructs the attack from the web access log and PCAP, corroborates against the SIEM and authentication logs, characterises the consequence, and then delivers a small detection ruleset addressing the techniques observed.

In general, the assessment includes the following phases:

01. Web access log analysis

- Reconstruct the request sequence from web_access.log (the authoritative record)
- Identify the injection probe, column counting, and UNION exploitation chain
- Anchor each step to a precise timestamp (UTC+02:00)



02. PCAP response analysis

- Follow HTTP Stream on suspect conversations (Wireshark / tshark)
- Recover server responses to confirm what data left the database
- Identify the exfiltration moment and the largest response (full users table)



03. SIEM and authentication review

- Review wazuh-alerts.json for corroborating signals
- Correlate the auth.log SSH events against the attacker IP and the dump time
- Consolidate indicators of compromise across all sources



04. Consequence analysis

- Establish the credential reuse chain (dump, offline crack, SSH login)
- Assess the foothold and the exposure of every stored credential
- Map each finding to MITRE ATT&CK to enable downstream threat hunting



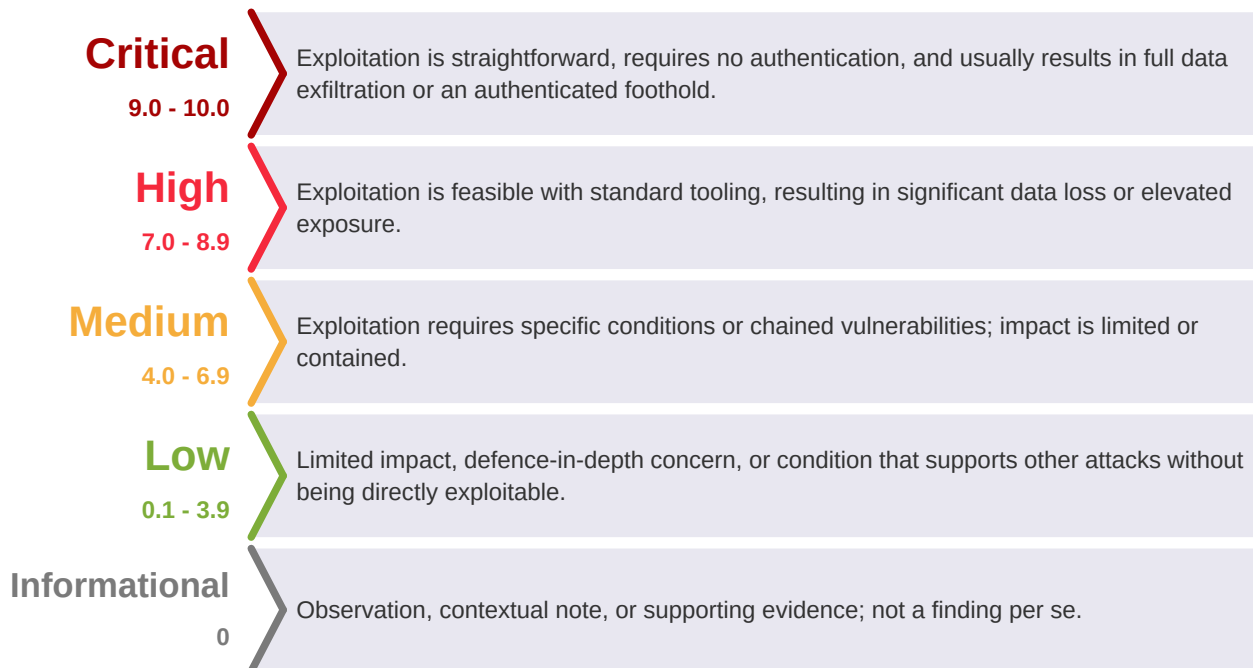
05. Detection engineering and reporting

- Author four Suricata signatures covering the SQL injection techniques observed
- Validate via offline PCAP replay (suricata -r attack.pcap) against a zero-alert baseline
- Consolidate findings into this report; analyst-rated severity classification



B. Severity Ratings

Severity is **analyst-rated**: the CVSS v3.1 base score is used as the foundation where a network attack vector exists, combined with the operational impact and the MITRE ATT&CK technique category. Finding 4.3 has no proper network attack vector of its own (the hash was recovered offline on the analyst workstation), so its severity is rated on operational impact and is marked *contextual* rather than carrying a CVSS base score.



Attack Timeline

Key anchors of the intrusion, in the web server local time zone (UTC+02:00). The five-hour gap between the database dump and the SSH login is consistent with offline cracking before credential reuse.

Time (+02:00)	Event
09:35:02	Error-based probe (single quote)
09:37:27	Column count confirmed (2) via ORDER BY 3 returning HTTP 500
09:38:29	Database name read: dvwa
09:39:58	Full users table exfiltrated (exfiltration anchor)
09:41:38	Blind boolean probing
14:48:06	Successful SSH login as j.martin from the attacker IP

Investigation Technical Summary

This section presents the four findings of the engagement. Findings are numbered 4.1 to 4.4 for parity with the published engagement repository. Severity is analyst-rated from the CVSS v3.1 base score (where a network vector exists) combined with the operational impact and the MITRE ATT&CK technique category. Every claim is anchored to the evidence bundle (web access log, PCAP, SIEM export, auth log).

4.1 SQL Injection on the id parameter

Severity:

CRITICAL

Finding type:

SQL Injection (UNION-based, in-band)

CVSS v3.1 base score:

9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

CWE:

CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)

MITRE ATT&CK:

T1190 (Exploit Public-Facing Application)

Affected scope:

- bru-web-01, endpoint /dvwa/vulnerabilities/sqli/, parameter id

Description

The id parameter passes user input directly into a back-end SQL query without separating data from instructions. The attacker (172.16.50.10) confirmed and then exploited the flaw, escalating from a simple probe to full database read access through the public-facing portal.

Evidence

The attack chain is visible in web_access.log in the following order:

- 09:35:02 error-based probe: id=1', then 1'', then 1' OR '1'='1. A lone single quote breaks the query and forces a database error, confirming the field is injectable.
- 09:37:24 to 09:37:27 column counting: ORDER BY 1 and 2 return HTTP 200, ORDER BY 3 returns HTTP 500. The query returns exactly two columns.
- 09:37:28 onward UNION exploitation: UNION SELECT NULL, NULL, then @@version, database(), and information_schema queries to map the engine, the database, and the users table.

Impact

Full read access to the back-end database through the public-facing portal. This single flaw is the root cause of findings 4.2, 4.3, and 4.4.

Recommendation:

- Replace the vulnerable query with parameterised queries (prepared statements) so user input can never be parsed as SQL.
- Add server-side input validation on the `id` parameter (numeric allowlist).
- Adopt least-privilege database accounts so the web user cannot read `information_schema` or other databases.

References and articles:

CWE-89: <https://cwe.mitre.org/data/definitions/89.html>

OWASP A03:2021 Injection: https://owasp.org/Top10/A03_2021-Injection/

MITRE ATT&CK T1190: <https://attack.mitre.org/techniques/T1190/>

4.2 Database enumeration and full users table exfiltration

Severity:

HIGH

Finding type:

Sensitive data exposure (database exfiltration)

CVSS v3.1 base score:

7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

CWE:

CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)

MITRE ATT&CK:

T1213 (Data from Information Repositories)

Affected scope:

- dvwa database on bru-web-01

Description

Using the confirmed UNION injection, the attacker read the database name and dumped the complete users table, exposing every username and password hash.

Evidence

Server responses recovered from the PCAP via Follow HTTP Stream. TCP stream 14 returns the database name dvwa. TCP stream 447 (UNION SELECT user,password FROM users) returns the full table, response size 5745, the largest of the session. Six accounts were exposed:

Username	MD5 hash	Note
admin	5f4dcc3b5aa765d61d8327deb882cf99	MD5 of password
gordonb	e99a18c428cb38d5f260853678922e03	application default
1337	8d3533d75ae2c3966d7e0d4fcc69216b	application default
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	application default
smithy	5f4dcc3b5aa765d61d8327deb882cf99	MD5 of password
j.martin	ccf5538dc31d435d6bab145c924041d8	NexaCorp employee account

Exfiltration time: 2026-05-30 09:39:58 (+02:00). This is the moment the data left the database.

Impact

Every credential stored in the portal database is exposed. The account of operational concern is j.martin, the only one matching a real employee naming pattern and the only one the attacker re-queried by name.

Recommendation:

- Treat every account in the dumped table as compromised and force a password reset.
- Fix the underlying injection that allowed the read (see finding 4.1).
- Apply least-privilege so the web account cannot read the full users table.

References and articles:

CWE-200: <https://cwe.mitre.org/data/definitions/200.html>

MITRE ATT&CK T1213: <https://attack.mitre.org/techniques/T1213/>

4.3 Weak credential recovered offline

Severity:

HIGH

Finding type:

Weak password storage and weak credential (offline recovery)

CVSS v3.1 base score:

Contextual (offline cracking, no network attack vector)

CWE:

CWE-916 (Use of Password Hash With Insufficient Computational Effort), CWE-521 (Weak Password Requirements)

MITRE ATT&CK:

T1110.002 (Brute Force: Password Cracking)

Affected scope:

- account j.martin
-

Description

The MD5 hash of j.martin was recovered offline against the standard rockyou.txt wordlist using John the Ripper, on the analyst workstation, never against the live server. The recovery took under one second.

Evidence

Hash ccfc5538dc31d435d6bab145c924041d8 resolves to a cleartext password that satisfies a typical complexity policy (upper case, lower case, digit, special character, eleven characters).

Analysis

The password falls instantly because the pattern is common and present in standard wordlists. Syntactic complexity is not the same as resistance to guessing. The MD5 storage compounds the problem: it is unsalted and fast, so an exposed hash is recovered at negligible cost.

Recommendation:

- Block common and predictable passwords by checking new passwords against a known-breached list.
- Enforce multi-factor authentication so a single recovered password is not sufficient to authenticate.
- Migrate password storage away from unsalted MD5 toward a slow, salted hash (bcrypt or argon2).

References and articles:

CWE-916: <https://cwe.mitre.org/data/definitions/916.html>

CWE-521: <https://cwe.mitre.org/data/definitions/521.html>

MITRE ATT&CK T1110.002: <https://attack.mitre.org/techniques/T1110/002/>

4.4 Credential reuse, successful SSH foothold

Severity:**CRITICAL**

Finding type:Credential reuse leading to authenticated remote access

CVSS v3.1 base score:9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

CWE:CWE-308 (Use of Single-factor Authentication), CWE-307 (Improper Restriction of Excessive Authentication Attempts)

MITRE ATT&CK:T1078 (Valid Accounts), T1021.004 (Remote Services: SSH)

Affected scope:

- bru-web-01 (SSH service)
-

Description

The stolen and cracked credential was reused against SSH on the same server. The login succeeded, giving the attacker an authenticated foothold under a legitimate employee identity.

Evidence

From auth.log:

```
2026-05-30T14:48:06 bru-web-01 sshd[14736]: Accepted password for  
j.martin from 172.16.50.10 port 49145 ssh2  
session opened for user j.martin(uid=1001)
```

The successful login at 14:48:06 came from the same IP used for the SQL Injection. Immediately after, the attacker attempted to spray other dumped accounts (j.martin retry, admin, gordonb), all of which failed. The gap between the dump (09:39:58) and the successful SSH login (14:48:06) is about five hours, consistent with offline cracking before reuse.

Impact

A confirmed interactive foothold on the server under a real employee identity. This access is the entry point for follow-on activity.

Recommendation:

- Disable or reset j.martin immediately and terminate any active sessions.
- Enforce MFA on SSH and add lockout or rate limiting against credential spraying.
- Restrict SSH to the internal network or VPN; block 172.16.50.10 at the perimeter.

References and articles:

CWE-308: <https://cwe.mitre.org/data/definitions/308.html>

CWE-307: <https://cwe.mitre.org/data/definitions/307.html>

MITRE ATT&CK T1078: <https://attack.mitre.org/techniques/T1078/>

MITRE ATT&CK T1021.004: <https://attack.mitre.org/techniques/T1021/004/>

Indicators of Compromise

The following indicators were consolidated from the evidence bundle. A SIEM-ingestible list is maintained alongside the engagement repository.

Indicator	Value
Attacker IP	172.16.50.10
Targeted endpoint	/dvwa/vulnerabilities/sqli/, parameter id
Payload patterns	%27, ORDER BY, UNION SELECT, database(), information_schema, FROM users, SUBSTRING(password, ...)
Compromised account	j.martin (uid 1001)
Consequence	Successful SSH login, 2026-05-30 14:48:06 (+02:00)

Remediation Recommendations

P0 - Immediate (within hours)

- Disable or reset `j.martin` and terminate active sessions. The attacker holds valid SSH access.
- Block `172.16.50.10` at the perimeter.
- Force a password reset for every account in the dumped users table.

P1 - Short term (within days)

- Fix the root cause: parameterised queries on the `id` parameter, plus input validation.
- Harden SSH: MFA, restrict to internal or VPN, lockout against spraying.
- Enforce a real password policy that blocks common and breached passwords.

P2 - Medium term

- Migrate password storage from unsalted MD5 to bcrypt or argon2.
- Deploy detection (IDS or WAF rules) that alert on SQL Injection patterns in the request URI. Four validated Suricata signatures covering the techniques seen in this incident are provided (see Detection Engineering).
- Review network segmentation: external SSH to `bru-web-01` should not be reachable.

Detection Engineering

A. Methodology and validation

After the investigation reconstructed the attack, four Suricata signatures were written to detect each SQL injection technique observed, then validated by replaying the capture offline (`suricata -r attack.pcap`). The host had no default ruleset, so the baseline was zero alerts and every count below comes from these four local rules. Each rule inspects the normalized `http.uri` buffer, so URL-encoded payloads are matched in decoded form.

B. Rule catalog and alert counts

SID	Detection	Technique	MITRE ATT&CK	Alerts
1000001	UNION SELECT in URI	UNION-based extraction	T1190	16
1000002	Single quote probe	Error-based probing	T1190	44
1000003	Blind boolean (SUBSTRING)	Boolean blind injection	T1190	18
1000004	information_schema enum	Schema enumeration	T1213	2

C. Tuning notes

The single-quote probe fires most often because the apostrophe is the common opening token across nearly every injection variant. These rules are lab-tuned for a training portal where any SQL keyword in a URI is suspicious. Production deployment would need tighter scoping (source and destination, `flow:established`) and narrower content matches on generic keywords such as `SUBSTRING`. The full ruleset, alert counts, and false-positive notes are maintained in the engagement repository.

Risk Matrix

The following 5x5 risk matrix plots each finding by its **Impact** (vertical axis) against its **Likelihood** (horizontal axis). Findings in the upper-right quadrant (high impact, high likelihood) require the most urgent remediation.

Impact / Likelihood	Very Low (1)	Low (2)	Medium (3)	High (4)	Very High (5)
Very High (5)				4.4	4.1
High (4)				4.3	4.2
Medium (3)					
Low (2)					
Very Low (1)					

Critical Risk	High Risk	Medium Risk	Low Risk
---------------	-----------	-------------	----------

Critical-risk cluster (top-right quadrant): 4.1, 4.2, 4.4. The SQL Injection (4.1) is the root cause, trivially exploitable and unauthenticated; the exfiltration (4.2) exposes every stored credential; the SSH foothold (4.4) is a confirmed authenticated entry point. Finding 4.3 (weak credential, offline recovery) sits just below in the high-impact band: it is the hinge that turns the data exposure into the live foothold.

MITRE ATT&CK Mapping

Each finding is mapped to one or more MITRE ATT&CK Enterprise techniques (v15) to align this assessment with industry-standard threat intelligence frameworks. This mapping enables downstream integration with SIEM detection rules and threat hunting playbooks.

Finding	Tactic	Technique ID	Technique Name
4.1	Initial Access	T1190	Exploit Public-Facing Application
4.2	Collection	T1213	Data from Information Repositories
4.3	Credential Access	T1110.002	Brute Force: Password Cracking
4.4	Initial Access	T1078	Valid Accounts
4.4	Lateral Movement	T1021.004	Remote Services: SSH

Tools and Standards

A. Tools Used During the Engagement

The following tools were used to investigate, validate, and document the findings in this report. All operations were conducted offline against the provided evidence bundle; no tool was run against any live production system.

Category	Tool	Purpose
Web log analysis	grep / sed / awk	Reconstruct the request sequence from web_access.log and isolate the injection chain
Network capture analysis	Wireshark / tshark	Follow HTTP Stream, recover server responses, identify the exfiltration response
Credential analysis	John the Ripper	Offline MD5 recovery against rockyou.txt, on the analyst workstation only
	rockyou.txt wordlist	Standard wordlist used for the offline recovery
SIEM and log review	Wazuh (alerts export)	Corroborating signal review from wazuh-alerts.json
	auth.log review	Confirm the successful SSH login and the failed spraying attempts
Detection engineering	Suricata 6.0.4	Author and validate four SQL injection signatures via offline PCAP replay
Remote access	OpenSSH + Tailscale	Remote access to the BeCode lab workstation
Documentation	VS Code, Markdown	Note-taking, journal, report drafting

B. Standards and Frameworks Followed

This engagement was conducted in alignment with internationally recognized incident response, web security, and detection engineering frameworks to ensure methodological rigor and consistent finding classification.

Standard / Framework	Version	Application in This Report
NIST SP 800-61 Rev. 2	2012	Computer Security Incident Handling Guide - investigation methodology
NIST SP 800-86	2006	Integrating Forensic Techniques into Incident Response - evidence handling
NIST SP 800-94	2007	Guide to Intrusion Detection and Prevention Systems - signature design
SANS PICERL	-	Incident response phases (Preparation, Identification, Containment, Eradication, Recovery, Lessons)
OWASP Top 10	2021	A03:2021 Injection - root vulnerability classification
MITRE ATT&CK Enterprise	v15	Technique mapping for findings and detection rules
CVSS - Common Vulnerability Scoring System	v3.1	Base-metric severity scoring where a network attack vector exists
CWE - Common Weakness Enumeration	v4.16	Root-cause classification of each finding
Suricata rule syntax	6.0.4	Detection rule authoring (SIDs 1000001-1000004)

Glossary and Acronyms

The following table defines the technical acronyms and terms used throughout this report.

Acronym	Full Name	Description
API	Application Programming Interface	Endpoints exposed for programmatic interaction
argon2	Argon2 password hash	Modern slow, salted password hashing function
bcrypt	Blowfish-based hash	Slow, salted password hashing function resistant to fast cracking
CVE	Common Vulnerabilities and Exposures	Public catalogue of disclosed vulnerabilities
CVSS	Common Vulnerability Scoring System	Numerical severity scoring framework
CWE	Common Weakness Enumeration	Classification of software weaknesses
DBMS	Database Management System	Software managing the back-end database
DFIR	Digital Forensics and Incident Response	Investigation discipline for security incidents
DVWA	Damn Vulnerable Web Application	Deliberately vulnerable PHP/MySQL training application
HTTP	HyperText Transfer Protocol	Application-layer protocol for the web
IDS	Intrusion Detection System	Sensor that alerts on suspicious traffic
IoC	Indicator of Compromise	Artifact pointing to malicious activity
JSON	JavaScript Object Notation	Lightweight data-interchange format
JtR	John the Ripper	Offline password recovery tool
MD5	Message Digest 5	Fast, unsalted hash, unsuitable for password storage
MFA	Multi-Factor Authentication	Requires more than one factor to authenticate
MITRE ATT&CK	Adversarial Tactics, Techniques and Common Knowledge	Threat intelligence framework
NIDS	Network Intrusion Detection System	Sensor placed on the network for traffic inspection
NIST	National Institute of Standards and Technology	U.S. agency publishing security standards
OWASP	Open Worldwide Application Security Project	Community publishing web security guidance
PCAP	Packet Capture	File format storing captured network traffic
PICERL	Prep, Identification, Containment, Eradication, Response, and Lessons Learned	Security incident response process
PoC	Proof of Concept	Demonstration that a vulnerability is exploitable
rockyou.txt	-	Common breached-password wordlist used for offline cracking
SID	Signature Identifier	Unique numeric ID of a Suricata or Snort rule
SIEM	Security Information and Event Management	Log aggregation and correlation platform
SOC	Security Operations Center	Team responsible for security monitoring and response
SQL	Structured Query Language	Language used to query relational databases
SQLi	SQL Injection	Injecting SQL through unsanitised input to alter a query
SSH	Secure Shell	Encrypted remote-administration protocol
TCP	Transmission Control Protocol	Connection-oriented transport-layer protocol
uid	User Identifier	Numeric identity of a Unix account
UNION	SQL UNION operator	Combines result sets, abused to read arbitrary columns
URI	Uniform Resource Identifier	String identifying a resource
UTC	Coordinated Universal Time	Time reference; PCAP window is expressed in UTC
WAF	Web Application Firewall	Filter that inspects and blocks malicious web requests
Wazuh	-	Open-source SIEM and HIDS platform

BeCode Corp.

DFIR / SOC Training

Reference: BCC-2026 | INC-2026-004

Classification: Confidential

Analyst

Johan-Emmanuel Hatchi

BeCode Cybersecurity Bootcamp - Promotion 2025-2026

linkedin.com/in/johan-emmanuel-hatchi

github.com/Jhatchi